

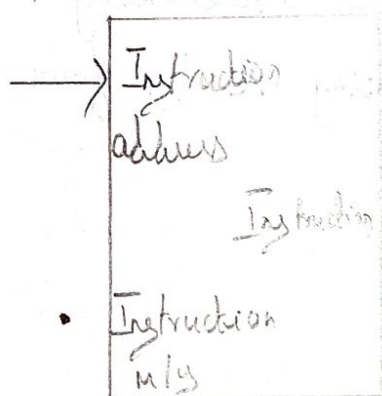
Design Convention of a Processor - Building a MIPS Datapath and designing a Control unit - Execution of a Complete Instruction - Hardwired and Micro Programmed Control - Introduction to Multicore - Graphics Processing units - Case Study - NVIDIA GPU.

BUILDING A DATAPATH

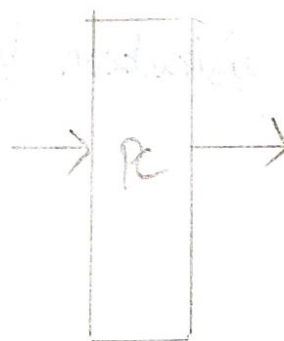
In order to design a datapath we must list the major components required to execute each class of MIPS instructions. Components required to form a datapath is known as datapath element.

Datapath Element:-

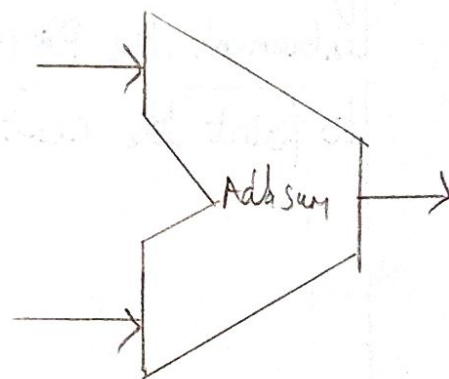
* It is a unit to operate on or hold data within a processor. In the MIPS implementation the datapath elements include the instruction and data memories, the register file, ALU and adders.



a) instruction memory



b) Program Counter



c) Adder

* Instruction Memory

* Program Counter

The Instruction memory needs only to provide read access because the datapath does not write instruction.

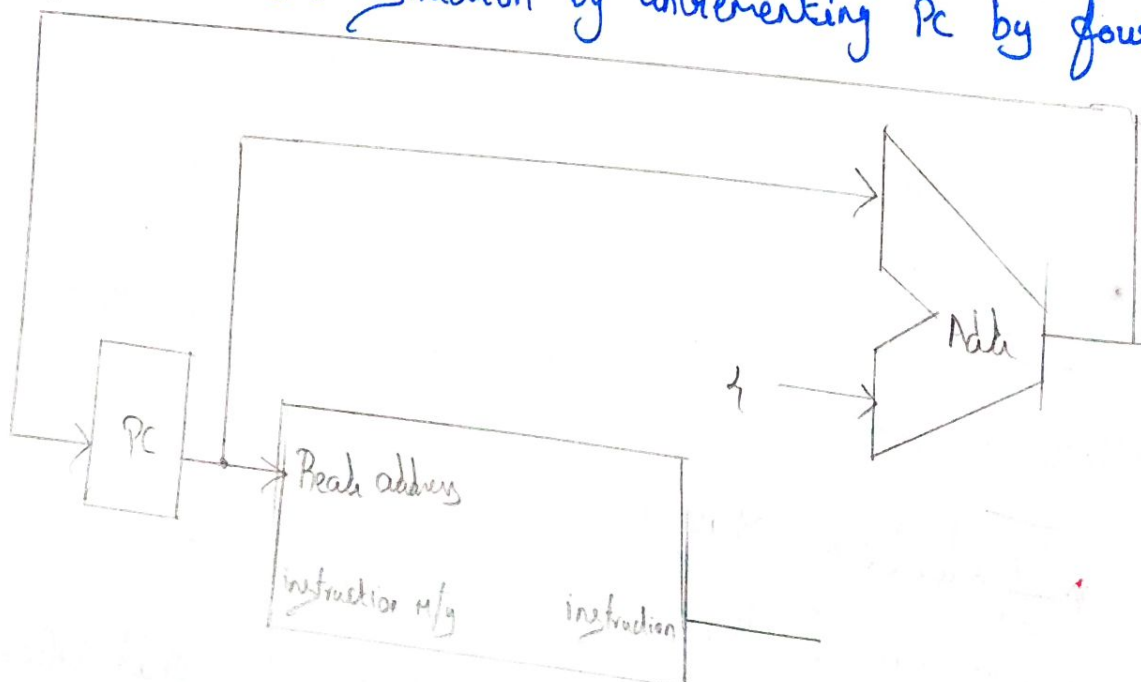
Instruction Memory $\rightarrow$ is called as Combinational element because it will perform only read. the output at any time reflects the contents of the location specified by the address. i/p is no read control signal is needed.

Program Counter $\rightarrow$ It is a 32 bit register that is written at the end of every clock and they does not need a Write Control Signal. PC is a register containing the address of the instruction in the Program being executed.

Adder $\rightarrow$ is a combinational element used to add two 32 bit i/p and place the sum on its o/p.

FETCHING PHASE:-

To execute any instruction, we must start by fetching the instructions from m/y. To prepare for executing the next instruction, we have to increment the Program Counter. The purpose of incrementing PC is to point the next instruction by incrementing PC by four Bytes



Arithmetic Logic Instructions:-

35

* It is also called as R-format instruction. To Perform ALU operation these instructions read two registers and write the registers and write the result to one register.

* It Performs the operation on the contents of the registers and this instruction includes add, Sub, AND, OR and slt.

* Processors having 32 general Purpose registers and Some Special Purpose registers are stored in Separate Space of mips. 32 general Purpose registers are stored in a Structure called a Register file.

Register file:-

* It is a State element that contains a Set of registers that can be read and written by supplying a Register number to be accessed. Register file contains the registers State of the computer.

R-TYPE INSTRUCTION OPERAND:-

* R-format instructions has three register operands to Perform ALU operation, two registers to read data and one register to write data.

* To read data word from the registers we have to specify the register number.

* It Specifies which data has to be read from which registers present in the register file.

* To write a data word into register we need two ip.

* One to Specify the register number to be written

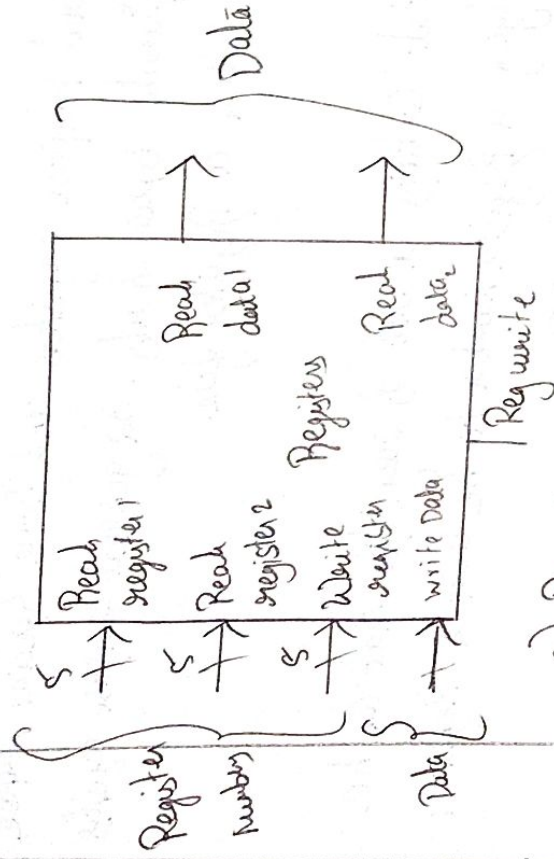
* One to supply the data to be written into the register.

Write operations are controlled by the write control signal and must be asserted for a write to occur at the clock edge.

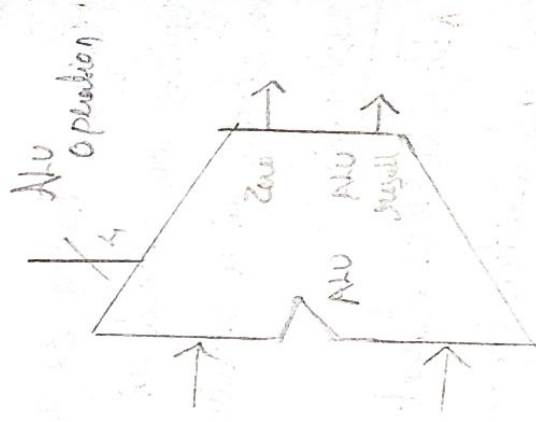
There are two elements we need to implement the R-format ALU operations such as

* Register

* ALU



a) Registers



b) ALU

BRANCH INSTRUCTIONS:-

There are two kinds of branch instructions are available

* beq - branch equal

* bneq - branch unequal

The beq instruction has three operands, in that

* two operands are registers used to compare equality

* one operand is a 16 bit offset used to compute the branch target address relative to the branch instruction address.

* beq, \$t1, \$t2, offset

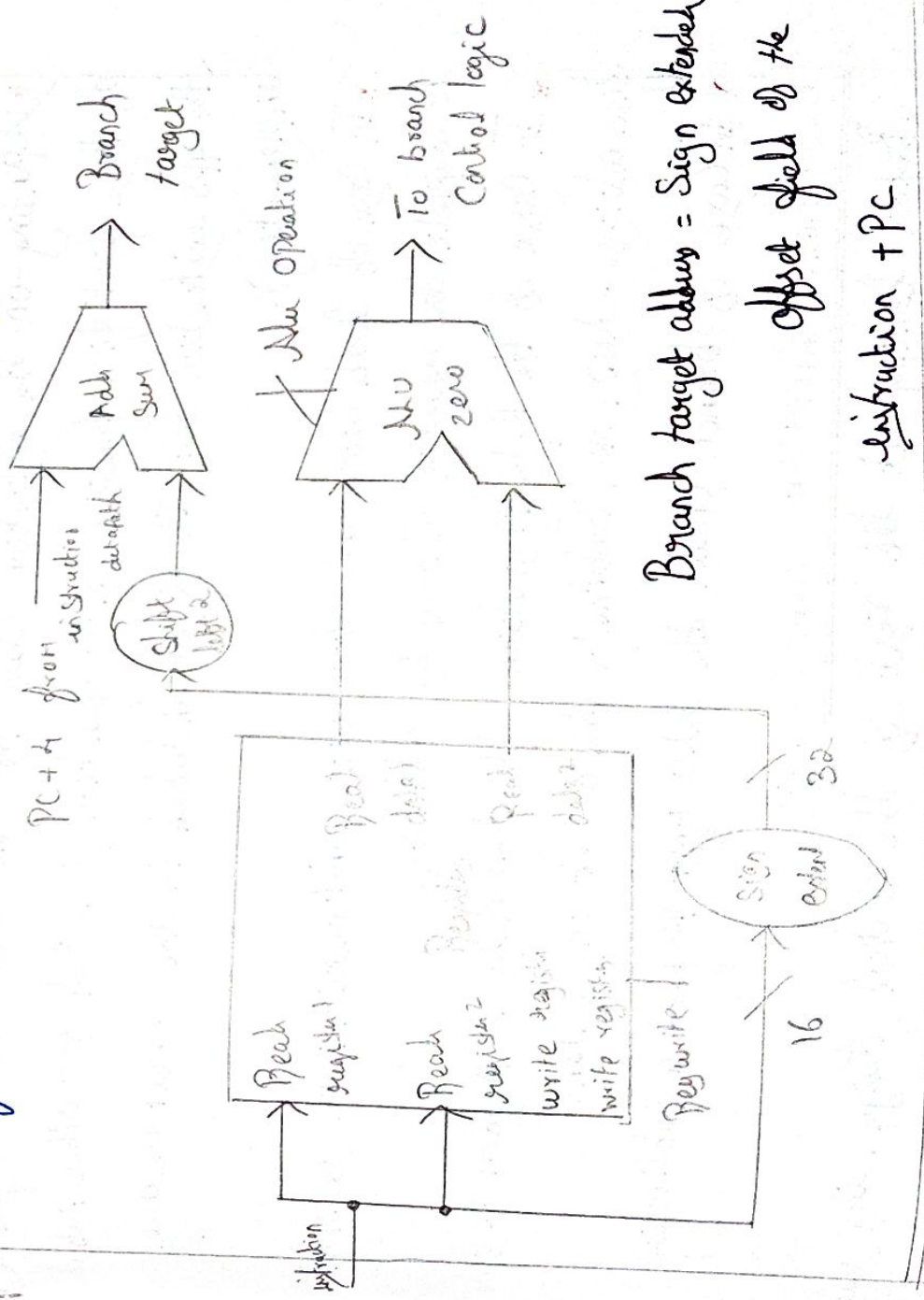
Branch target Address...

- * It's an address specified in branch, which becomes the new PC if the branch is taken.
- * If the operands are equal, the branch target address becomes the new PC and it is called as branch taken.
- * if the operands are not equal, the incremented PC should replace the current PC and it is called as branch is not taken.
- * Branch data path will perform two kinds of operations.

- * Compute the branch target address
- * Compute the register contents.

To compute the branch target address, the branch data path includes a sign extension unit.

To perform Compare, we need to use the register file.



DESIGNING A CONTROL UNIT

We restrict ourselves to implement load word (lw), Store word (sw), branch equal (beq) and the arithmetic logical instruction add, sub, AND, OR, and set on less than. We will also the design to include a jump instruction.

ALU Control:-

The MIPS ALU defines the six following combination of four control i/p's:

ALU Controlling	Function
0000	AND
0001	OR
0010	Add
0110	Subtract
0111	Set on less than
1100	NOR

ALU op	Action
00	Load and Store
01	Subtract for beq
10	The operation encoded in the function field
11	-

* Depending on the instruction class the ALU will need to perform one of these first five function (NOR function is needed for other parts of the MIPS instruction set. It is not included in the subset we are implementing).

* In case of load word and store word instruction, we use the ALU to complete the memory address by addition.

* In case of the R-type instruction, the ALU needs to perform one of the five actions - AND, OR, Subtract, add or set on less than.

* In case of branch equal, the ALU must perform a subtraction.

DESIGNING THE MAIN CONTROL UNIT:-

Before looking at the rest of the control design, it is used to review the formats of the three instruction classes.

OpCode	rs	rt	rd	Shamt	func
0	rs	rt	rd	Shamt	func
31:26	25:21	20:16	15:11	10:6	5:0

OpCode is 0. These instructions have three register operands: rs, rt and rd. rs and rt are sources, and rd is the destination. The funct field is an R-type function defined in the previous section.

Format for load & store instructions:

OpCode	rs	rt	address
31:26	25:21	20:16	15:0

Load or Store. The register rs is the base register that is added to the 16 bit address field to form the memory address. For loads, rt is the destination register for the load value, for stores, rt is the source register whose value should be stored into memory.

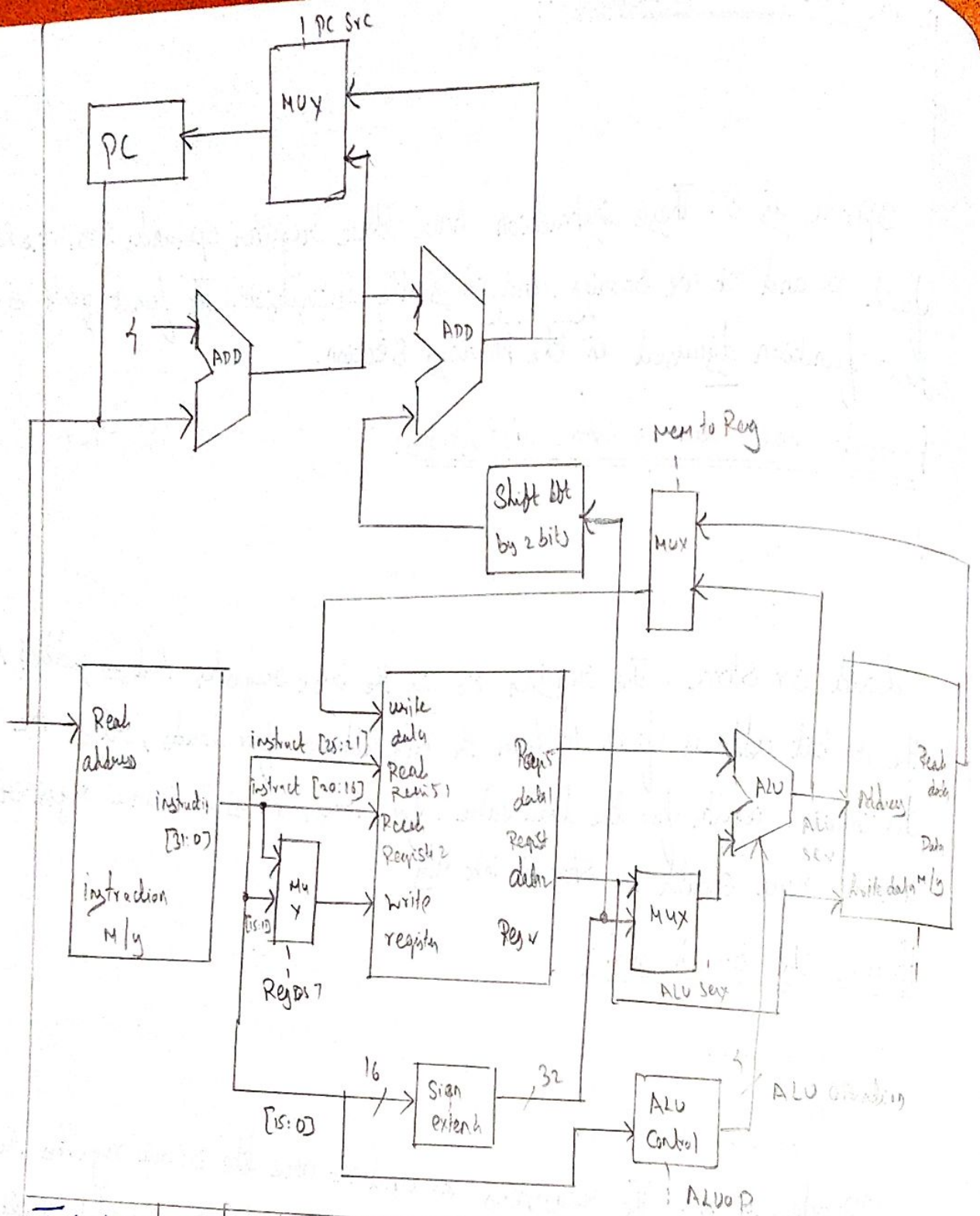
Format for branch equal:

OpCode	rs	rt	address
4	rs	rt	address
31:26	25:21	20:16	15:0

OpCode is 4. The registers rs and rt are the source registers that are compared for equality. The 16 bit address field is sign extended, shifted and added to the PC+4 to compute the branch target address.

IMPORTANT OBSERVATIONS ABOUT THE INSTRUCTION FORMAT:

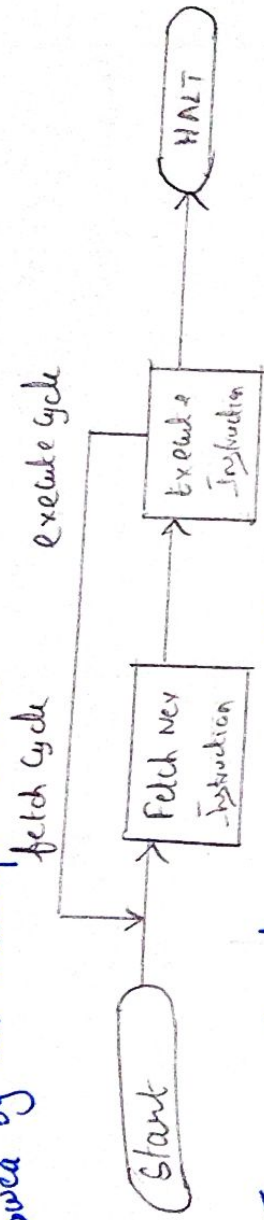
We can add the instruction labels and extra multiplexers to the simple datapath. These additions plus the ALU control block, the write signals for state elements, the read signal for the data memory, and the control signals for the multiplexers.



Instruction	Regdt	ALUSrc	MemtoReg	Regw	MemR	MemW	Branch	ALUop1	ALUop2
R format	1	0	0	1	0	0	0	0	0
lw	0	1	1	1	0	0	0	0	0
sw	X	1	X	0	0	0	0	0	0
beq	X	0	X	0	0	0	1	0	1

EXECUTION OF A COMPLETE INSTRUCTION

* It consists of an instruction fetch, followed by zero or more operand fetches, execution, followed by zero or more operand store, followed by an interrupt check.



* The Major Computer System Components (Processor, Main M/Y, I/O Modules) need to be interconnected in order to exchange data & Control signals.

* The Most Popular means of interconnection is the use of a shared system bus consisting of multiple lines.

* The CPU exchanges data with M/Y. The CPU typically makes use of internal registers: MAR, MDR

INSTRUCTION FETCH AND EXECUTE::

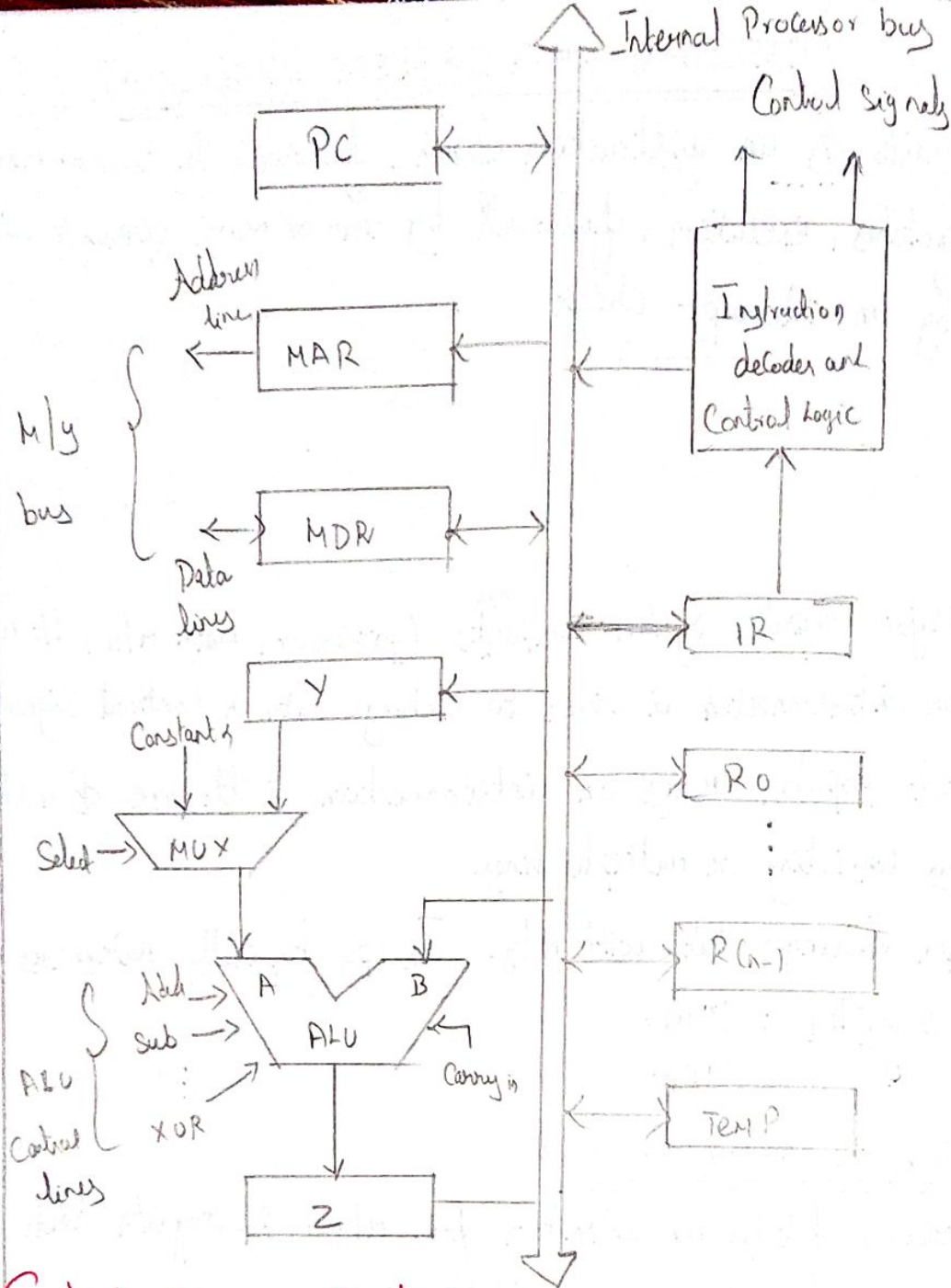
* The Processor fetches an instruction from M/Y - PC register holds the address of the instruction to be fetched next.

* The Processor increments the PC after each instruction fetch so that it will fetch the next instruction in the sequence - unless told otherwise.

* The fetched instruction is loaded into the instruction register (IR) in the Processor. The instruction contains bits that specify the action the Processor will take.

* The Processor interprets the instruction and performs the required action.

* Assume the next instruction is stored at (Current instruction address + 4)



Control Sequence of fetch cycle

Step Action

- 1 Pout ARin Read Set Carry in of ALU clear of Add, zin
- 2 Zout, PCin WMFC (Wait for memory completion)
- 3 DRout, IRin

Control Sequence of Execution cycle

Eg.: Add R1, X

Step Action

- 4 Address of IRout, ARin, Read

5. R_{out}, Y_{in}, WMFC
6. DR_{out}, Add, Z_{in}
7. Z_{out}, R_{lin}
8. End

HARDWIRED & MICRO PROGRAMMED CONTROL UNIT

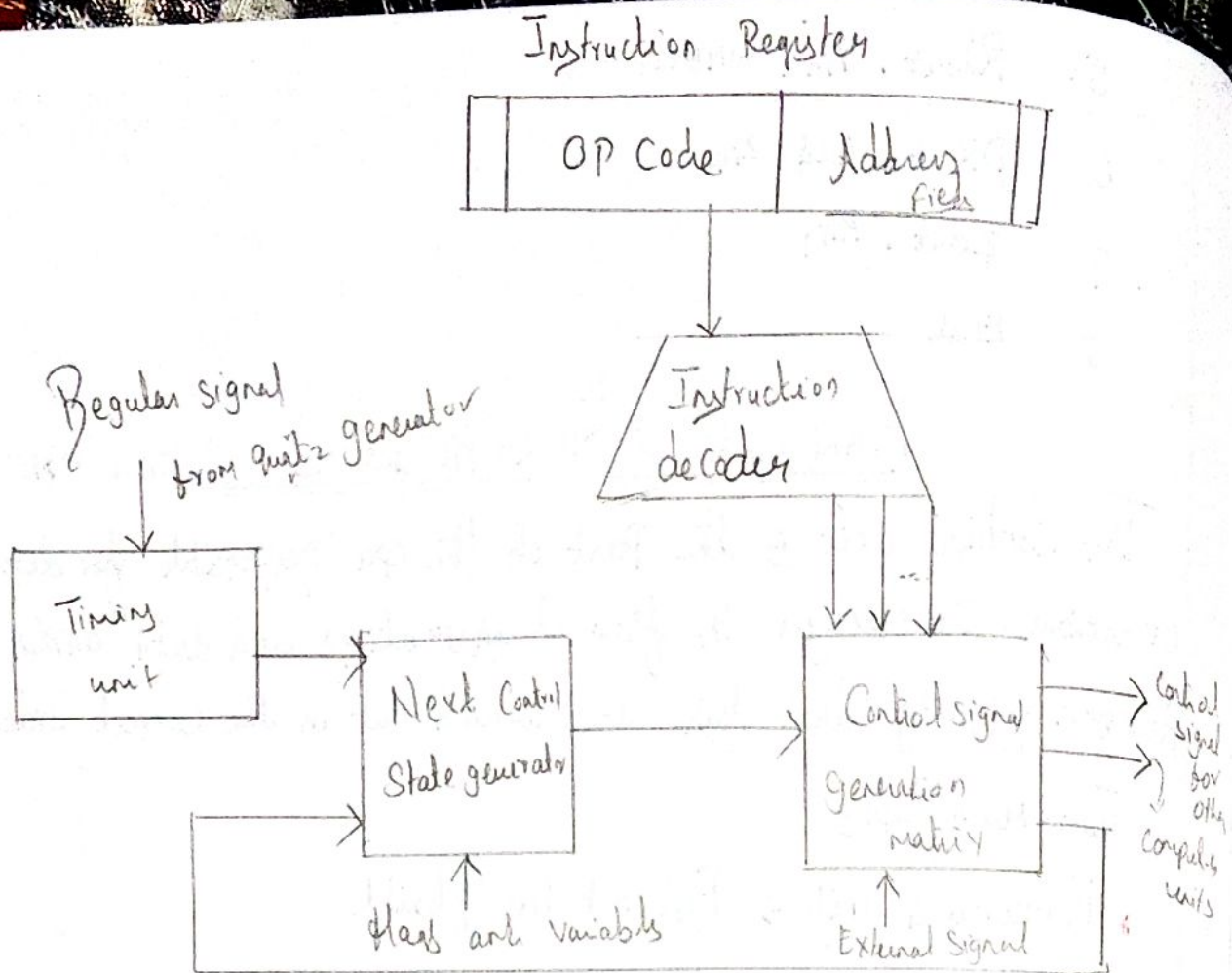
The Control unit is the part of the CPU responsible for directing operations. It manages the flow of instructions and data within the processor, ensuring that tasks are carried out in the correct order.

Two main types

- * Hardwired units → Fast but less flexible
- * Microprogrammed units → flexible but slower

HARDWIRED CONTROL UNIT :-

- * It can be viewed as a state machine that changes from one state to another in every clock cycle, depending on the contents of the instruction register, the Condition Codes, & the external i/p.
- * The o/p of the state machine are the Control Signals. The sequence of the operation carried out by this machine is determined by the wiring of the logic elements and hence is named "hardwired".
- * fixed logic circuits that correspond directly to the Boolean expressions are used to generate the Control Signals.
- * Hardwired Control is faster than microprogrammed control.
- * A Controller that uses this approach can operate at high speed.
- * RISC architecture is based on the hardwired control unit.



MICROPROGRAMMED CONTROL UNIT:

- * The Control Signals associated with operations are stored in Special Mly units inaccessible by the programmer as Control words.
- * Control Signals are generated by a program that is similar to Machine language programs.
- * The Micro programmed Control unit is slower in speed because of the time it takes to fetch microinstructions from the Control Mly.

Some Important Terms:

- * Control word: - is a word whose individual bit represent various Control signals.
- * Micro-routine: - A sequence of Control word corresponding to the Control sequence of a machine instruction constitutes the micro routine for that instruction.

40
* Micro instruction $\rightarrow$ Individual Control words in this micro-routine are referred to as microinstructions.

* Micro program $\rightarrow$ A sequence of micro instruction is called a micro program.

* which is stored in a ROM or RAM called a Control M/y (CM).

* Control Store $\rightarrow$ the micro routines for all instructions in the instruction set of a computer are stored in a special M/y called the Control store.

TYPES OF MICRO PROGRAMMED CONTROL UNIT:

Based on the type of Control word stored in the Control M/y (CM), it is classified into two types.

HORIZONTAL MICRO PROGRAMMED CONTROL UNITS:-

The Control signals are represented in the decoded binary format that is bit/cb. Eg if 53 Control signals are present in the Processor then 53 bits are required. More than 1 Control signal can be enabled at a time.

- * Supports longer Control words
- * is used in Parallel Processing applications.
- * allow a higher degree of parallelism. if degree is n , n Cs are enabled at a time.

* It requires no addition h/w. Means it is faster than vertical

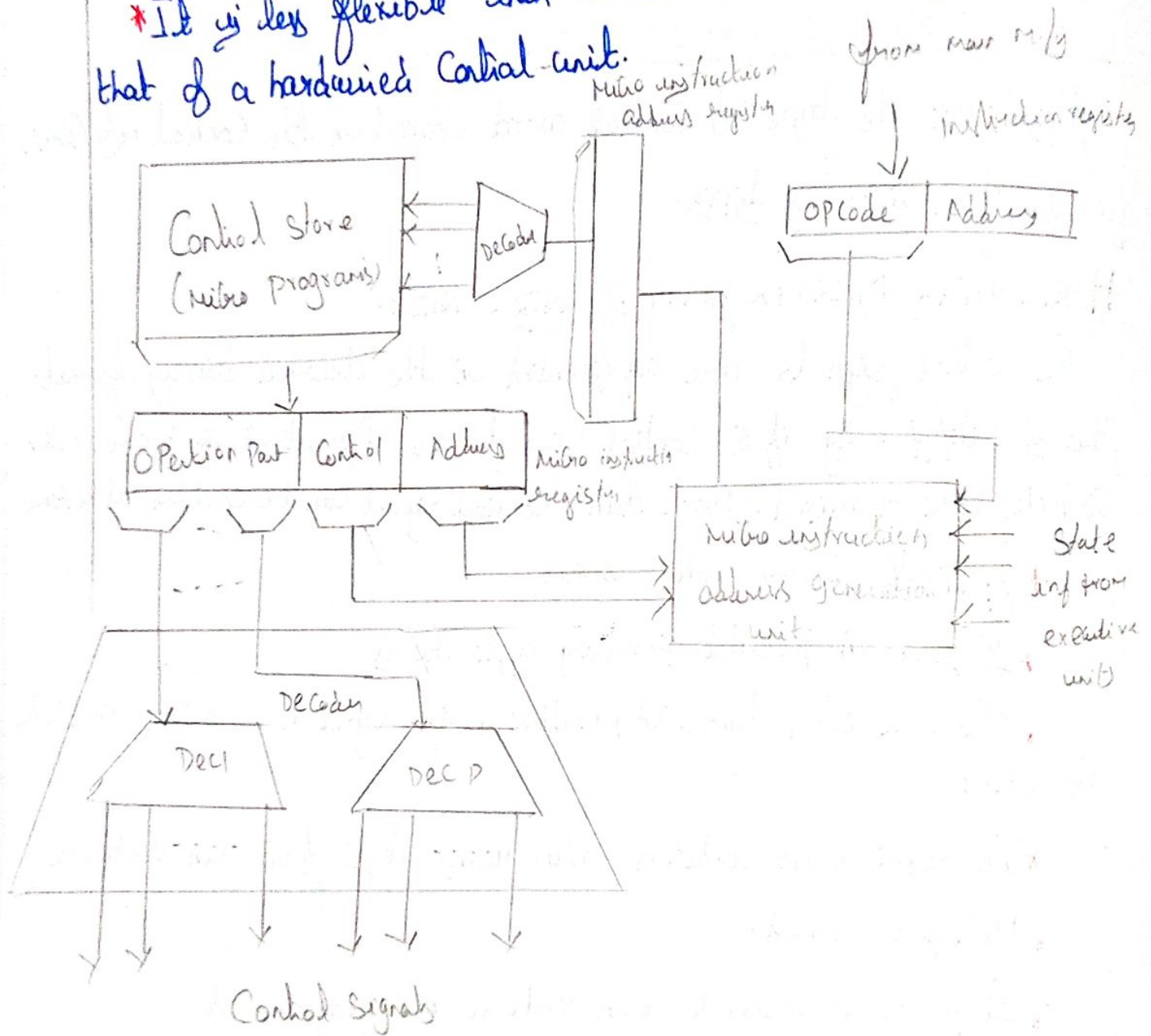
* Microprogrammed.

* It is more flexible than vertical microprogrammed.

VERTICAL MICRO PROGRAMMED CONTROL UNIT:-

Control signals are represented in the encoded binary format. for N Control signals $\log_2(N)$ bit are required.

- * It Supports shorter Control words.
- * It Supports easy implementation of new Control Signals though it is more flexible.
- * It allow a low degree of Parallelism i.e the degree of Parallelism is either 0 or 1.
- * It is less flexible than horizontal but more flexible than that of a hardwired Control unit.



MULTICORE

41

* A Multicore Computer, combines two or more processors on a single computer chip

* The use of more complex single processor chips has reached a limit due to hardware performance issues.

* The multicore architecture poses challenges to S/W developers to exploit the capability for multithreading across multiple cores.

* The main variables in a multicore organization are the number of processors on the chip, the number of levels of cache m/y and the extent to which cache m/y is shared.

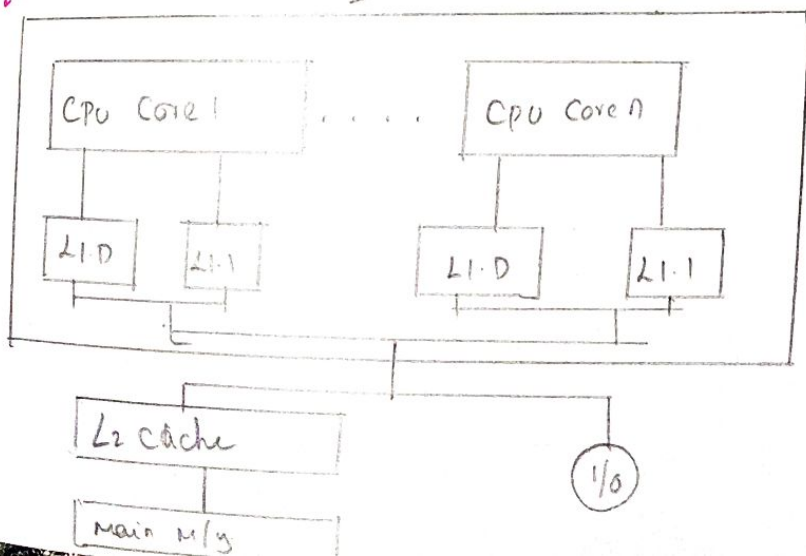
* It also known as a chip multiprocessor combines two or more processors (called cores) on a single piece of silicon (called a die).

* Each core consists of all the components of an independent processor, such as register ALU, pipeline h/w and control unit, L1 instruction and data caches.

* In addition to the multiple cores, contemporary multicore chips also include L2 cache and in some can L3 cache.

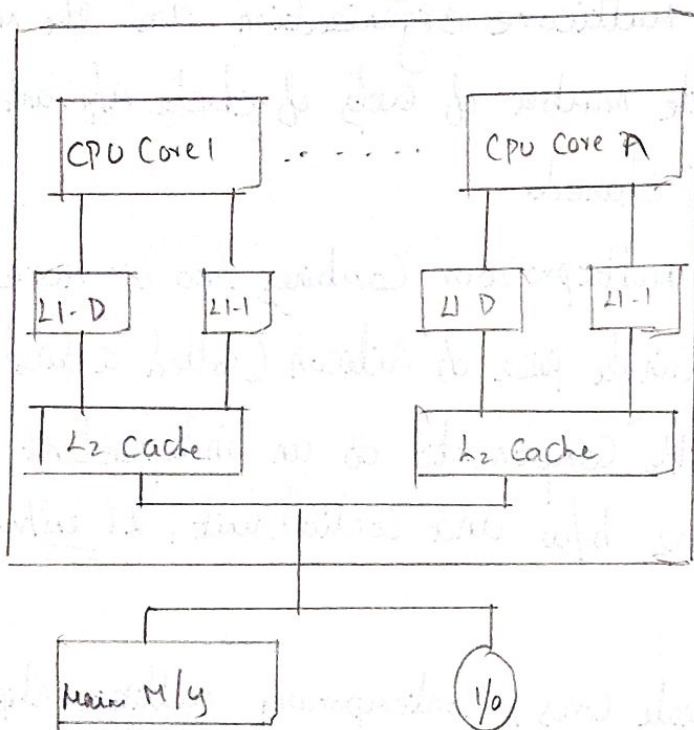
ORGANIZATION:-

Dedicated L1 Cache

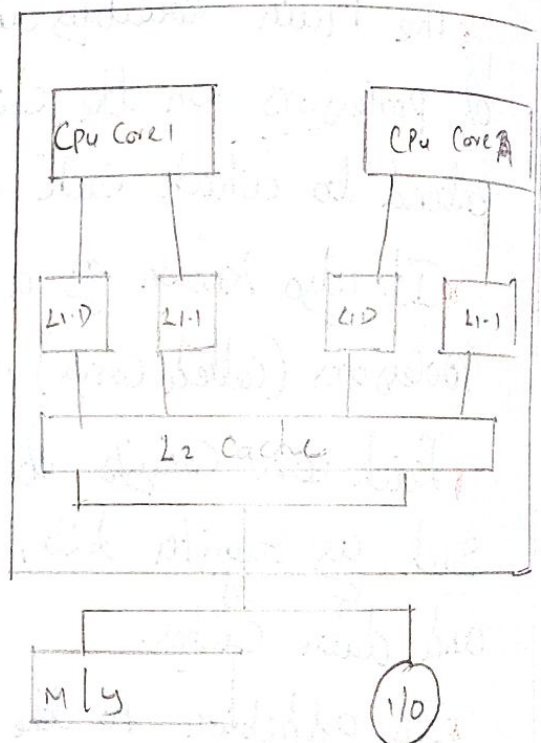


* In this organization, the only on chip cache is L1 cache, each core having its own dedicated L1 cache. The L1 cache is divided into instruction and data caches. The organization is also one in which there is no on-chip cache sharing.

* In this there is enough area available on the chip to allow for L2 cache. An eg of this organization is the AMD operation. The intel core duo has the organization. The amount of cache memory available on the chip continues to grow, Performance Considerations dictate splitting off a separate

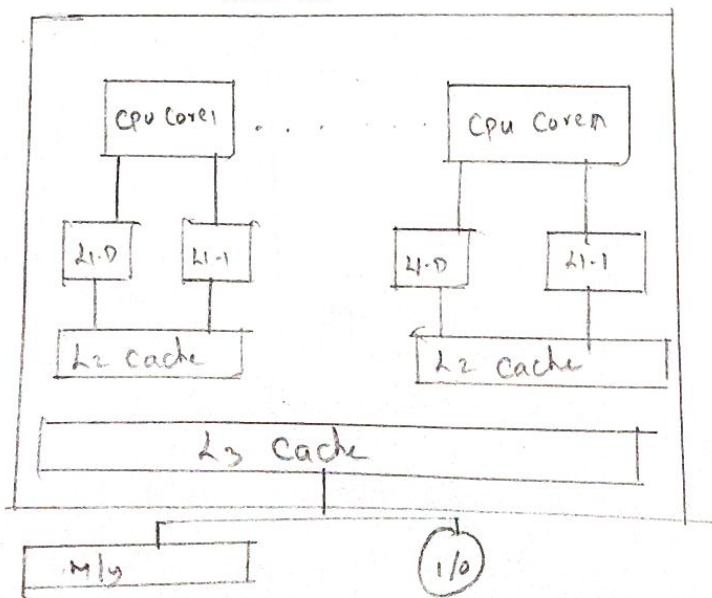


Dedicate L2 cache



Shared L2 cache

Shared L3 cache



Intel has introduced a number of multicore products in recent years

- * The Intel Core Duo
- * The Intel Core i7

Performance:

Hardware Issues:

* IT systems have experienced a steady, exponential increase in execution performance for decades.

* This increase is due to refinement in the question of organization of the processor on the chip and the increase in the clock frequency.

* Increase in parallelism

* Pipelining

* Superscalar

* Simultaneous multithreading (SMT)

* Power Consumption.

Software Performance Issues:

* A detailed examination of the S/W performance issues related to multicore organization is a huge task.

* A number of classes of application benefit directly from the ability to scale throughput with the number of cores.

* Multithreaded ~~un~~ native applications.

* Multiprocess applications

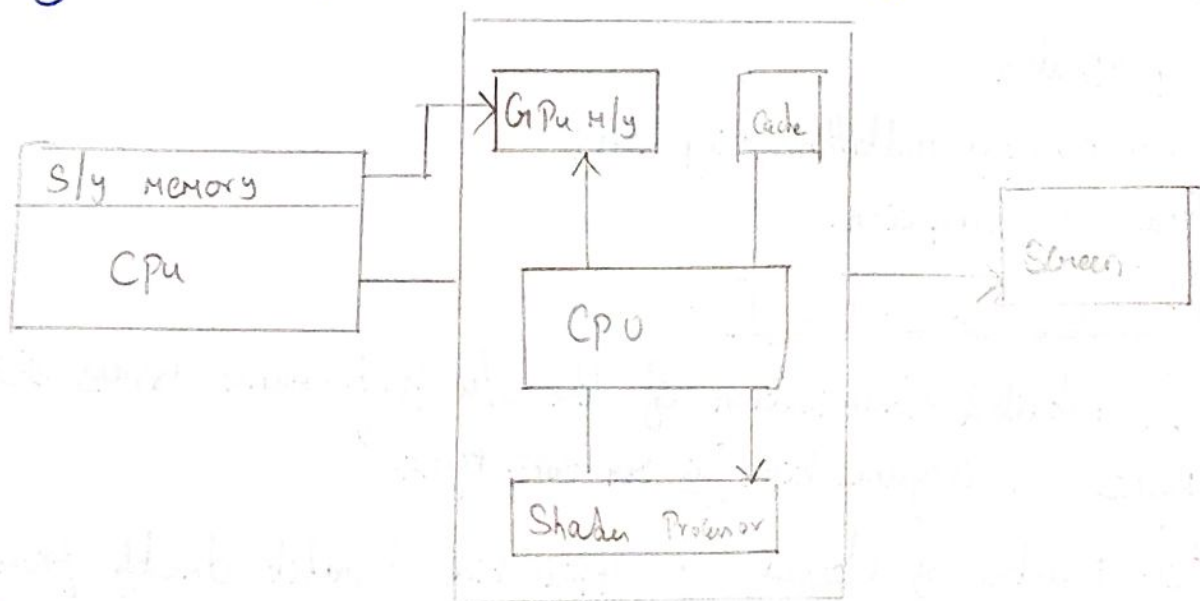
* Java applications.

GRAPHICS PROCESSING UNIT

It has evolved significantly over the years, making computers much better at handling images and complex tasks. The newest Graphics Processing unit have opened up new opportunities in many areas, like gaming, Machine learning & Content Creation.

* Main function → A GPU accelerates rendering of images, video & animation by handling many calculations in parallel. It's also used in machine learning and scientific computing for the same reason.

* Massive Parallelism → GPUs have thousands of small cores (like CUDA cores for NVIDIA, Stream Processors for AMD) that can process many data streams simultaneously, ideal for parallel workloads.



CORE COMPONENTS:

- * GPU die → Central processing unit of the graphics card that houses all compute units.
- * VRAM → dedicated high speed m/y that stores graphics data and textures for fast access.
- * Cooling system → Keep the GPU from overheating during operation.

Power Regulation & Interface: Provide Power and Connect the CPU to the Motherboard & Monitor.

Features of GPU:-

- * Rendering 2D & 3D Graphics
- * Supporting advanced SLI
- * Cross Device functionality
- * Handling Complex Color Processing
- * Accelerating AI & Machine Learning.

NVIDIA GPU

* It has improved its GPU architecture gradually since its beginning. In the beginning of GPU development graphics rendering occupied most attention with simple design for streaming multiprocessors. The company intensified its effort to build better parallel computing feature after the applications grew beyond their original display purpose.

* It appears in each product update to provide stronger processing for harder jobs. More CUDA Cores at every generation helps NVIDIA handle bigger AI and HPC application demands.

* It keeps introducing better memory technologies to ensure faster performance through HBM & GDDR memory. The new solution helps GPU process data faster to eliminate memory restrictions.

* Unified memory

* High Speed Interconnects